



Managing the build and the configuration





Default build organization

- ▶ All the build output goes into a directory called `output/` within the top-level Buildroot source directory.
 - `0 = output`
- ▶ The configuration file is stored as `.config` in the top-level Buildroot source directory.
 - `CONFIG_DIR = $(TOPDIR)`
 - `TOPDIR = $(shell pwd)`
- ▶ `buildroot/`
 - **`.config`**
 - `arch/`
 - `package/`
 - **`output/`**
 - `fs/`
 - ...



Out of tree build: introduction

- ▶ Out of tree build allows to use an output directory different than `output/`
- ▶ Useful to build different Buildroot configurations from the same source tree.
- ▶ Customization of the output directory done by passing `O=/path/to/directory` on the command line.
- ▶ Configuration file stored inside the `$(O)` directory, as opposed to inside the Buildroot sources for the in-tree build case.
- ▶ `project/`
 - `buildroot/`, Buildroot sources
 - `foo-output/`, output of a first project
 - `.config`
 - `bar-output/`, output of a second project
 - `.config`



Out of tree build: using

▶ To start an out of tree build, two solutions:

- From the Buildroot source tree, simply specify a `0=` variable:

```
make 0=../foo-output/ menuconfig
```

- From an empty output directory, specify `0=` and the path to the Buildroot source tree:

```
make -C ../buildroot/ 0=$(pwd) menuconfig
```

- ▶ Once one out of tree operation has been done (`menuconfig`, loading a defconfig, etc.), Buildroot creates a small wrapper `Makefile` in the output directory.
- ▶ This wrapper `Makefile` then avoids the need to pass `0=` and the path to the Buildroot source tree.



Out of tree build: example

1. You are in your Buildroot source tree:

```
$ ls arch board boot ... Makefile ... package ...
```

2. Create a new output directory, and move to it:

```
$ mkdir ../foobar-output  
$ cd ../foobar-output
```

3. Start a new Buildroot configuration:

```
$ make -C ../buildroot O=$(pwd) menuconfig
```

4. Start the build (passing `O=` and `-C` no longer needed thanks to the wrapper):

```
$ make
```

5. Adjust the configuration again, restart the build, clean the build:

```
$ make menuconfig  
$ make  
$ make clean
```



Full config file vs. *defconfig*

- ▶ The `.config` file is a *full* config file: it contains the value for all options (except those having unmet dependencies)
- ▶ The default `.config`, without any customization, has 4742 lines (as of Buildroot 2024.02)
 - Not very practical for reading and modifying by humans.
- ▶ A *defconfig* stores only the values for options for which the non-default value is chosen.
 - Much easier to read
 - Can be modified by humans
 - Can be used for automated construction of configurations



defconfig: example

- ▶ For the default Buildroot configuration, the *defconfig* is empty: everything is the default.
- ▶ If you change the architecture to be ARM, the *defconfig* is just one line:

```
BR2_arm=y
```

- ▶ If then you also enable the `stress` package, the *defconfig* will be just two lines:

```
BR2_arm=y BR2_PACKAGE_STRESS=y
```



Using and creating a *defconfig*

- ▶ To use a *defconfig*, copying it to `.config` is not sufficient as all the missing (default) options need to be expanded.
- ▶ Buildroot allows to load *defconfig* stored in the `configs/` directory, by doing:
`make <foo>_defconfig`
 - It overwrites the current `.config`, if any
- ▶ To create a *defconfig*, run: `make savedefconfig`
 - Saved in the file pointed by the `BR2_DEFCONFIG` configuration option
 - By default, points to `defconfig` in the current directory if the configuration was started from scratch, or points to the original *defconfig* if the configuration was loaded from a *defconfig*.
 - Move it to `configs/` to make it easily loadable with `make <foo>_defconfig`.



Existing *defconfigs*

- ▶ Buildroot comes with a number of existing *defconfigs* for various publicly available hardware platforms:
 - RaspberryPi, BeagleBone Black, CubieBoard, Microchip evaluation boards, Minnowboard, various i.MX6 boards
 - QEMU emulated platforms
- ▶ List them using `make list-defconfigs`
- ▶ Most built-in *defconfigs* are minimal: only build a toolchain, bootloader, kernel and minimal root filesystem.

```
$ make qemu_arm_vexpress_defconfig  
$ make
```

- ▶ Additional instructions often available in `board/<boardname>`, e.g.: `board/qemu/arm-vexpress/readme.txt`.
- ▶ Your own *defconfigs* can obviously be more featureful



Assembling a *defconfig* (1/2)

- ▶ *defconfigs* are trivial text files, one can use simple concatenation to assemble them from fragments.

platform1.frag

```
BR2_arm=y  
BR2_TOOLCHAIN_BUILDROOT_WCHAR=y  
BR2_GCC_VERSION_7_X=y
```

platform2.frag

```
BR2_mipsel=y  
BR2_TOOLCHAIN_EXTERNAL=y  
BR2_TOOLCHAIN_EXTERNAL_CODESOURCERY_MIPS=y
```

packages.frag

```
BR2_PACKAGE_STRESS=y  
BR2_PACKAGE_MTD=y  
BR2_PACKAGE_LIBCONFIG=y
```



Assembling a *defconfig* (2/2)

debug.frag

```
BR2_ENABLE_DEBUG=y BR2_PACKAGE_STRACE=y
```

Build a release system for *platform1*

```
$ ./support/kconfig/merge_config.sh platform1.frag packages.frag  
$ make
```

Build a debug system for *platform2*

```
$ ./support/kconfig/merge_config.sh platform2.frag packages.frag  
    debug.frag  
$ make
```

- ▶ Saving fragments is not possible; it must be done manually from an existing *defconfig*



Other building tips

▶ Cleaning targets

- Cleaning all the build output, but keeping the configuration file:

```
$ make clean
```

- Cleaning everything, including the configuration file, and downloaded file if at the default location:

```
$ make distclean
```

▶ Verbose build

- By default, Buildroot hides a number of commands it runs during the build, only showing the most important ones.
- Passing `V=1` also applies to packages, like the Linux kernel, busybox...
- To get a fully verbose build, pass `V=1`:

```
$ make V=1
```